CSCE 411: Design and Analysis of Algorithms

Week 14: Randomized Algorithms: Fingerprinting, Hashing, PageRank

Date: April 21, 2026 *Nate Veldt, updated by Samson Zhou*

Course Logistics

- Randomized algorithms: Chapter 5
- Hashing and PageRank: Chapters 11 and 14
- Homework 5 out this weekend, due next Friday

1 Introduction to Randomized Algorithms

A **randomized algorithm** is defined by

An algorithm that always outputs the correct answer but has a random runtime is denoted a _____

We can also denote an algorithm with a fixed runtime but a probability of outputting an incorrect answer as a _____

If an algorithm has a failure probability of p, we can run it independently k times. The probability that it fails every single time is _____. The process of running an algorithm multiple times to reduce failure probability is:

The *expected* runtime of a Las Vegas algorithm is calculated over the _____

1.1 Classes of randomized algorithms

- Las Vegas:

- Monte Carlo:

1.2 Basic fingerprinting and verification structures

- A **fingerprint** is a small signature in which

- A **false positive** is a scenario in which

- **Freivalds' algorithm**: randomized approach to verify matrix multiplication $AB = C$:

$$\vec{r} \in \{0,1\}^n \text{ such that } A(B\vec{r}) = C\vec{r}$$

- **Polynomial Identity Testing**: checks if two polynomials are equivalent:

$$P(x), Q(x) \text{ where } P(r) = Q(r) \text{ for random } r \in S.$$

- **String Matching (Rabin-Karp)**: utilizes a rolling hash to compare strings:

$$H(P) = H(T[i \dots i + m - 1]) \text{ for some window } i.$$

- **Amplification**: a strategy in which repeating the algorithm decreases error probability.

1.3 Verification Problems using Fingerprinting

Many verification problems amount to evaluating identity over a randomized input.

Example 1. Matrix Multiplication. Given three matrices $A, B \in \mathbb{R}^{n \times n}$ and $C \in \mathbb{R}^{n \times n}$, verify if $AB = C$.

Example 2. Database Equality. Let Alice and Bob hold databases D_A and D_B. Find if they are identical with minimal communication.

Example 3. Find Substrings. Return the starting indices of a pattern P in text T:

2 Hashing Representation

Consider a dynamic set of elements S from a fixed universe of keys $U = \{1, 2, \ldots, |U|\}$.

Hash Table The **hash table** $\mathcal{T}$ of size m is defined so that

Hash Function The **hash function** $h(x)$ of $\mathcal{T}$ is

Hashing Activity

Consider the following hash function $h(x) = x \pmod 7$:

- Write down the hash value for $x = 10$
- Write down the hash value for $x = 24$
- Write down the load factor if $n = 14$ elements are inserted
- Write down the elements that map to slot 3
- Find the number of collisions among the keys $\{3, 10, 17\}$

Question 1. *Assume that U is a large universe of keys and we map them to a table of size m. Let n be the number of elements inserted. How much worst-case time is needed to search the table if an adversary picks the keys?*

A $O(1)$

B $O(\log n)$

C $O(n)$

D $O(mn)$

3 Universal Hashing Algorithm

Dictionary Problem: Given a universe U and hash table T, map elements $x \in S$ into T evenly, preventing an adversary from causing $\Omega(n)$ worst-case search times.

We will do this using the *universal hashing* scheme.

Attribute	Explanation	Initialization
h.family	tells us whether a hash function belongs to a	
x.key		
x.next		

We will also make use of

Basic Idea

- Define a family of hash functions $\mathcal{H}$
- Randomly *select* $h \in \mathcal{H}$ at the start of execution
- Continuously update

Example

`hashing1.png`

hashing2.png

3.1 Collisions and Universal Hash Families

Definition 1. Given a universe U, table size m, and a randomly selected $h \in \mathcal{H}$, a _____ is a set of functions where

It is furthermore strongly universal if it generates a unique random permutation

from U to m that is _____

Benefits of Universal Hashing

- If chaining is used, it provides an _____
- It tells us the _____
- It provides a _____

```
UNIVERSAL-HASH-INSERT(T, x)
  Initialize H as universal family
  if T.hash_func == NIL then
      T.hash_func = RandomSelect(H)
  end if
  h = T.hash_func
  index = h(x.key)
  if T[index] == NIL then
      T[index] = x
  else
      curr = T[index]
      while curr.next ≠ NIL do
          curr = curr.next
      end while
      curr.next = x
  end if
  x.next = NIL
```

3.2 Code and Runtime Analysis

- We assume T uses chaining for collision resolution.
- Initializing the hash function takes _____
- Each element x enters the table, and hashing it takes _____
- When we *insert* x, we traverse up to _____

Using expected value analysis, what is the overall runtime of this method?

4 PageRank Algorithm

Unlike regular directed graph traversal, the PageRank algorithm:

- Explores the *stationary distribution* of a random walk before stabilizing and outputting probability scores
- All nodes in the graph are scored simultaneously (rather than just paths from a single node s)
- We keep track of a global *damping factor*, and each node is associated with two buffers for when it is *updated* and *finalized*.

Each node $u \in V$ is associated with the following attributes

Attribute	Explanation	Initialization
u.out_degree	tells us whether a node has outgoing links to be *distributed*, *absorbed*, and *teleported*	
u.PR	probability mass when u is evaluated	
u.new_PR	probability mass when u is finished being updated	
u.Adj	outgoing links/"destinations" of u	

```
PageRank(G, d, k)
  for v ∈ V do
      v.new_PR = 0
      v.PR = 1/|V|
  end for
  for step = 1 to k do
      for u ∈ V do
          PR-Update(G, u, d)
      end for
      for v ∈ V do
          v.PR = v.new_PR
          v.new_PR = 0
      end for
  end for
```

```
PR-Update(G, u, d)
  teleport = (1 − d)/|V|
  u.new_PR = u.new_PR + teleport
  deg = u.out_degree
  if deg > 0 then
      for v ∈ u.Adj do
          v.new_PR = v.new_PR + d · (u.PR/deg)
      end for
  else
      Handle sink by distributing evenly
      for v ∈ V do
          v.new_PR = v.new_PR + d · (u.PR/|V|)
      end for
  end if
```

pagerank_walk.png

4.1 Runtime Analysis

Question 2. *What is the runtime of k iterations of PageRank, assuming that we store the graph in an adjacency list, and assuming there are no sinks (deg > 0 for all nodes)?*

A $O(k|V|)$

B $O(k|E|)$

C $O(k(|V| + |E|))$

D $O(k|V|^2)$

E $O(k|E|^2)$

4.2 Properties of PageRank

Theorem 4.1. *In any execution of PageRank on a web graph $G = (V, E)$, with a damping factor $0 < d < 1$, exactly one of the following conditions holds for the underlying Markov chain:*

- *The chain is regular and* _____
- *The distribution of probability mass across V* _____
- *The initial scores assigned to nodes* _____

4.3 Classification of Web Graph Structures

Given a web graph $G = (V, E)$ running PageRank models a random walk traversing the edges of E.

$$\hat{E} = \{(u, v) \colon u \in V \text{ and } v \in u.\text{Adj}\}$$

This is called the *transition space* of G.

Given any node $u \in V$, we can classify it based on the status of its outgoing edges when we are computing PageRank:

Structure	Explanation	How to resolve when evaluatin
Normal Node	node with outgoing links in $\hat{E}$	
Dead End	node u with no outgoing links	
Spider Trap	a group of nodes with no links to the outside	*Use* a dar factor $d < 1$
Disconnected Component	a subgraph with no incoming links from the rest of the web	*Use* ra teleportation with $1 - d$

5 Practice

manypageranks.png

Question 3. *How many of the above graphs contained a spider trap?*

A *1*

B *2*

C *3*

D *4*

E *none of them*